CENTRO UNIVERSITÁRIO DE ITAJUBÁ-FEPI

Curso de Sistemas De Informação

Andre Henrique Santos Souza, Matheus Henrique Menicalli da Rosa e João Gabriel Dos Santos Silva

Projeto: Sistema de Biblioteca (cadastro, autenticação, acervo, empréstimo, devolução, busca e relatórios)

ITAJUBÁ 2026

INTRODUÇÃO

Este documento define o planejamento de testes do Sistema de Biblioteca, com foco na validação

funcional e não funcional das principais operações de negócio: cadastro de usuários, autenticação,

cadastro de livros, empréstimo, devolução, busca de livros e emissão de relatórios.

O plano estabelece escopo, estratégia, tipos de teste, critérios de entrada e saída, recursos,

cronograma, casos de teste detalhados e registro de defeitos simulados. O objetivo é garantir

rastreabilidade, padronização e qualidade acadêmica na execução dos testes.

1.1 Objetivos do plano

• Definir o que será testado e com qual profundidade.

• Identificar riscos funcionais e de desempenho do sistema.

• Assegurar que os requisitos e regras de negócio foram implementados corretamente.

• Fornece evidências objetivas para avaliação de qualidade do software.

1.2 Escopo do sistema sob teste

Incluído no escopo:

• Cadastro de usuários.

• Login/autenticação.

• Cadastro de livros.

• Empréstimo de livros.

• Devolução de livros.

• Busca/consulta de livros disponíveis.

• Relatórios de livros emprestados e atrasos.

Fora de escopo nesta versão:

• Integrações externas (pagamento de multas, e-mail transacional).

• Persistência em banco real (quando rodando apenas o protótipo em memória).

• Controle avançado de perfis e permissões em múltiplos níveis.

2. Itens testáveis

Item	Descrição
Módulo de Usuários	Cadastro, validações, autenticação
Módulo de Acervo	Cadastro de livros e consulta
Módulo de Circulação	Empréstimo e devolução
Módulo de Relatórios	Empréstimos ativos e atrasos
Regras de Negócio	Limite de livros, prazo de devolução e consistência de estados

Requisitos funcionais principais

• RF01: Cadastrar usuário com nome, e-mail e senha válidos.

• RF02: Autenticar usuário por e-mail e senha.

• RF03: Impedir e-mails duplicados.

• RF04: Cadastrar livro com título e autor válidos.

• RF05: Permitir empréstimo apenas para usuário e livro válidos/disponíveis.

• RF06: Permitir devolução de empréstimo ativo.

• RF07: Exibir livros disponíveis.

• RF08: Exibir relatório de empréstimos ativos.

• RF09: Exibir relatório de atrasos.

Regras de negócio

• RN01: Usuário pode manter no máximo 3 livros emprestados simultaneamente.

• RN02: Prazo padrão de devolução é de 7 dias.

• RN03: Livro emprestado deve ser marcado como indisponível até devolução.

• RN04: Devolução duplicada deve ser rejeitada.

• RN05: Empréstimos em atraso devem constar no relatório de atrasos

Requisitos não funcionais (RNF)

•RNF01: Desempenho: operações críticas (`login`, `busca`, `empréstimo`, `devolução`) devem responder em até 2 segundos em 95% das execuções no ambiente acadêmico.

•RNF02: Confiabilidade: não deve haver inconsistência entre disponibilidade do livro e estado do empréstimo.

•RNF03: Usabilidade: mensagens de erro e sucesso devem ser claras, objetivas e orientadas à ação do usuário.

•RNF04: Segurança: o sistema não deve expor senha em logs, saídas de erro ou relatórios.

•RNF05: Portabilidade: execução compatível com Python 3.x e ambiente local padrão (Windows/Linux/macOS).

•RNF06: Manutenibilidade: código estruturado em funções/métodos com baixo acoplamento e alta legibilidade.

•RNF07: Rastreabilidade: todas as operações de empréstimo devem possuir ID único e vínculo com usuário/livro.

•RNF08: Testabilidade: funcionalidades devem permitir testes determinísticos e repetíveis via pytest.

3. Tipos de teste (com objetivo, aplicação e validação)

Teste de Unidade

Objetivo: validar, de forma isolada, métodos e funções do domínio (cadastro, login, empréstimo, devolução e validações).

Quando será aplicado: durante desenvolvimento e antes de integração de funcionalidades.

O que valida: regras de validação de entrada, retorno esperado, exceções corretas e consistência local do método.

```python
class TestCadastroUsuario:
    def test_cadastro_valido_retorna_id(self, biblioteca: Biblioteca):
        uid = biblioteca.cadastrar_usuario("João", "joao@test.com", "abcd")
        assert uid.startswith("U")
        u = biblioteca.obter_usuario(uid)
        assert u is not None
        assert u.nome == "João"
        assert u.email == "joao@test.com"

    def test_cadastro_email_duplicado(self, biblioteca: Biblioteca):
        biblioteca.cadastrar_usuario("Ana", "dup@email.com", "senha1234")
        with pytest.raises(ValueError, match="já cadastrado"):
            biblioteca.cadastrar_usuario("Bia", "dup@email.com", "outrasenha")

    def test_cadastro_nome_curto(self, biblioteca: Biblioteca):
        with pytest.raises(ValueError, match="Nome"):
            biblioteca.cadastrar_usuario("X", "x@email.com", "senha1234")

    def test_cadastro_email_invalido(self, biblioteca: Biblioteca):
        with pytest.raises(ValueError, match="E-mail"):
            biblioteca.cadastrar_usuario("Nome Ok", "sem_arroba", "senha1234")

    def test_cadastro_senha_curta(self, biblioteca: Biblioteca):
        with pytest.raises(ValueError, match="Senha"):
            biblioteca.cadastrar_usuario("Nome Ok", "ok@email.com", "123")
```

Teste de Integração

Objetivo: verificar a interação entre módulos (Usuários, Acervo e Circulação).

Quando será aplicado: após aprovação dos testes unitários principais.

O que valida: fluxo completo entre operações dependentes (ex.: cadastrar -> emprestar -> devolver -> atualizar disponibilidade).

```python
class TestEmprestimoEDevolucao:
    def test_emprestimo_remove_da_lista_disponiveis(
        self,
        biblioteca: Biblioteca,
        usuario_padrao: str,
        livro_padrao: str,
    ):
        biblioteca.emprestar(usuario_padrao, livro_padrao)
        ids = [l.id for l in biblioteca.consultar_livros_disponiveis()]
        assert livro_padrao not in ids

    def test_emprestimo_aparece_em_relatorio(
        self,
        biblioteca: Biblioteca,
        usuario_padrao: str,
        livro_padrao: str,
    ):
        biblioteca.emprestar(usuario_padrao, livro_padrao)
        emprestados = biblioteca.relatorio_livros_emprestados()
        assert len(emprestados) == 1
        assert emprestados[0].livro_id == livro_padrao

    def test_devolucao_libera_livro(
        self,
        biblioteca: Biblioteca,
        usuario_padrao: str,
        livro_padrao: str,
    ):
        eid = biblioteca.emprestar(usuario_padrao, livro_padrao)
        biblioteca.devolver(eid)
        ids = [l.id for l in biblioteca.consultar_livros_disponiveis()]
        assert livro_padrao in ids
```

```python
    def test_devolver_duas_vezes_erro(
        self,
        biblioteca: Biblioteca,
        usuario_padrao: str,
        livro_padrao: str,
    ):
        eid = biblioteca.emprestar(usuario_padrao, livro_padrao)
        biblioteca.devolver(eid)
        with pytest.raises(ValueError, match="já foi devolvido"):
            biblioteca.devolver(eid)

    def test_prazo_previsto_7_dias(
        self,
        biblioteca: Biblioteca,
        usuario_padrao: str,
        livro_padrao: str,
    ):
        base = datetime(2026, 3, 1, 10, 0, 0)
        eid = biblioteca.emprestar(usuario_padrao, livro_padrao, data_referencia=base)
        emp = biblioteca.obter_emprestimo(eid)
        assert emp is not None
        assert emp.data_devolucao_prevista == base + timedelta(days=7)
```

Teste de Sistema

Objetivo: avaliar o comportamento do sistema como um todo em cenário funcional completo.

Quando será aplicado: em ciclos de validação de versão candidata à entrega.

O que valida**:** aderência global aos requisitos funcionais e regras de negócio

```
(venv) C:\Users\mathe\Downloads\trabalhomalerba>pytest -v
========================================================= test session starts =========================================================
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\mathe\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\mathe\Downloads\trabalhomalerba
collected 23 items

test_sistema.py::TestCadastroUsuario::test_cadastro_valido_retorna_id PASSED                                                [  4%]
test_sistema.py::TestCadastroUsuario::test_cadastro_email_duplicado PASSED                                                  [  8%]
test_sistema.py::TestCadastroUsuario::test_cadastro_nome_curto PASSED                                                       [ 13%]
test_sistema.py::TestCadastroUsuario::test_cadastro_email_invalido PASSED                                                   [ 17%]
test_sistema.py::TestCadastroUsuario::test_cadastro_senha_curta PASSED                                                      [ 21%]
test_sistema.py::TestLogin::test_login_valido_retorna_id PASSED                                                             [ 26%]
test_sistema.py::TestLogin::test_login_senha_errada PASSED                                                                  [ 30%]
test_sistema.py::TestLogin::test_login_email_inexistente PASSED                                                             [ 34%]
test_sistema.py::TestCadastroLivro::test_cadastro_livro_valido PASSED                                                       [ 39%]
test_sistema.py::TestCadastroLivro::test_consulta_disponiveis_inclui_novo_livro PASSED                                      [ 43%]
test_sistema.py::TestEmprestimoEDevolucao::test_emprestimo_remove_da_lista_disponiveis PASSED                               [ 47%]
test_sistema.py::TestEmprestimoEDevolucao::test_emprestimo_aparece_em_relatorio PASSED                                      [ 52%]
test_sistema.py::TestEmprestimoEDevolucao::test_devolucao_libera_livro PASSED                                               [ 56%]
test_sistema.py::TestEmprestimoEDevolucao::test_devolver_duas_vezes_erro PASSED                                             [ 60%]
test_sistema.py::TestEmprestimoEDevolucao::test_prazo_previsto_7_dias PASSED                                                [ 65%]
test_sistema.py::TestRegrasDeNegocio::test_limite_tres_livros PASSED                                                        [ 69%]
test_sistema.py::TestRegrasDeNegocio::test_livro_indisponivel_nao_empresta PASSED                                           [ 73%]
test_sistema.py::TestRegrasDeNegocio::test_atraso_apos_prazo PASSED                                                         [ 78%]
test_sistema.py::TestRegrasDeNegocio::test_sem_atraso_dentro_do_prazo PASSED                                                [ 82%]
test_sistema.py::TestRegrasDeNegocio::test_usuario_inexistente_emprestimo PASSED                                            [ 86%]
test_sistema.py::TestRegrasDeNegocio::test_livro_inexistente_emprestimo PASSED                                              [ 91%]
test_sistema.py::TestValidacaoLivro::test_titulo_vazio PASSED                                                               [ 95%]
test_sistema.py::TestValidacaoLivro::test_autor_curto PASSED                                                                [100%]
```

Teste de Aceitação

Objetivo: confirmar que o sistema atende às necessidades do usuário e da biblioteca.

Quando será aplicado: ao final de cada ciclo principal, com roteiro de cenários de negócio.

O que valida: conformidade prática dos fluxos com expectativas do cliente/avaliador.

```python
class TestRegrasDeNegocio:
    def test_limite_tres_livros(
        self, biblioteca: Biblioteca, usuario_padrao: str
    ):
        l1 = biblioteca.cadastrar_livro("L1", "Autor")
        l2 = biblioteca.cadastrar_livro("L2", "Autor")
        l3 = biblioteca.cadastrar_livro("L3", "Autor")
        l4 = biblioteca.cadastrar_livro("L4", "Autor")
        biblioteca.emprestar(usuario_padrao, l1)
        biblioteca.emprestar(usuario_padrao, l2)
        biblioteca.emprestar(usuario_padrao, l3)
        with pytest.raises(ValueError, match="máximo"):
            biblioteca.emprestar(usuario_padrao, l4)

    def test_livro_indisponivel_nao_empresta(
        self, biblioteca: Biblioteca, usuario_padrao: str, livro_padrao: str
    ):
        outro = biblioteca.cadastrar_usuario("Outro", "outro@email.com", "senha1234")
        biblioteca.emprestar(usuario_padrao, livro_padrao)
        with pytest.raises(ValueError, match="não está disponível"):
            biblioteca.emprestar(outro, livro_padrao)

    def test_atraso_apos_prazo(
        self, biblioteca: Biblioteca, usuario_padrao: str, livro_padrao: str
    ):
        base = datetime(2026, 1, 1)
        biblioteca.emprestar(usuario_padrao, livro_padrao, data_referencia=base)
        limite = base + timedelta(days=Biblioteca.PRAZO_DEVOLUCAO_DIAS)
        atrasos = biblioteca.relatorio_atrasos(data_referencia=limite + timedelta(days=1))
        assert len(atrasos) == 1

    def test_sem_atraso_dentro_do_prazo(
        self, biblioteca: Biblioteca, usuario_padrao: str, livro_padrao: str
    ):
        base = datetime(2026, 2, 1)
        biblioteca.emprestar(usuario_padrao, livro_padrao, data_referencia=base)
        dentro = base + timedelta(days=3)
        assert biblioteca.relatorio_atrasos(data_referencia=dentro) == []

    def test_usuario_inexistente_emprestimo(self, biblioteca: Biblioteca):
        lid = biblioteca.cadastrar_livro("Título X", "Autor Y")
        with pytest.raises(ValueError, match="Usuário não encontrado"):
            biblioteca.emprestar("U9999", lid)

    def test_livro_inexistente_emprestimo(
        self, biblioteca: Biblioteca, usuario_padrao: str
    ):
        with pytest.raises(ValueError, match="Livro não encontrado"):
            biblioteca.emprestar(usuario_padrao, "L9999")
```

```python
class TestValidacaoLivro:
    def test_titulo_vazio(self, biblioteca: Biblioteca):
        with pytest.raises(ValueError, match="Título"):
            biblioteca.cadastrar_livro("", "Autor Válido")

    def test_autor_curto(self, biblioteca: Biblioteca):
        with pytest.raises(ValueError, match="Autor"):
            biblioteca.cadastrar_livro("Título Ok", "A")
```

Teste de Regressão

Objetivo: assegurar que alterações/correções não quebrem funcionalidades já validadas.

Quando será aplicado: após qualquer mudança em regras de negócio ou correção de defeitos.

O que valida: estabilidade funcional por meio da suíte automatizada completa.

```
    if len(senha) < 4:
        raise ValueError("Senha deve ter pelo menos 4 caracteres")
```

Para: Execução da suíte completa de testes após alterações no sistema.

```
    if len(senha) < 6:
        raise ValueError("Senha deve ter pelo menos 4 caracteres")
```

Resultado:

```
sistema.py:65: ValueError
=================================================== short test summary info ===================================================
FAILED test_sistema.py::TestCadastroUsuario::test_cadastro_valido_retorna_id - ValueError: Senha deve ter pelo menos 4 caractere
s
=============================================== 1 failed, 22 passed in 0.16s ==================================================
```

Teste de Desempenho

Objetivo: medir tempo de resposta das operações críticas em cenário acadêmico.

Quando será aplicado: após estabilidade funcional do sistema.

O que valida: se operações principais atendem limites aceitáveis de resposta.

```
(venv) C:\Users\mathe\Downloads\trabalhomalerba>pytest -q
.......................
23 passed in 0.05s
```

Estratégia e abordagem de testes

• Abordagem predominante: caixa preta para validação funcional dos requisitos e caixa branca em testes unitários.

• Técnica de execução: combinação de testes automatizados (pytest) e manuais exploratórios.

• Critério de priorização: risco de negócio (empréstimo/devolução/atrasos) e impacto ao usuário.

• Automação: suíte pytest -v para garantir repetibilidade e regressão

Teste de Sobrecarga

O teste de sobrecarga verifica como o sistema se comporta quando recebe uma quantidade de requisições maior do que o normal. O objetivo é identificar até que

ponto o sistema consegue funcionar corretamente e como ele reage quando atinge seu limite, podendo apresentar lentidão ou falhas.

```python
class TestSobrecarga:
    def test_sobrecarga_cadastro_e_emprestimo(self, biblioteca: Biblioteca):
        inicio = perf_counter()

        usuarios = []
        for i in range(200):
            uid = biblioteca.cadastrar_usuario(
                f"Usuario {i}",
                f"usuario{i}@email.com",
                "senha1234",
            )
            usuarios.append(uid)

        livros = []
        for i in range(600):
            lid = biblioteca.cadastrar_livro(f"Livro {i}", "Autor Sobrecarga")
            livros.append(lid)

        # Cada usuario pega 3 livros (regra de negocio mantida).
        idx = 0
        for uid in usuarios:
            for _ in range(3):
                biblioteca.emprestar(uid, livros[idx])
                idx += 1

        fim = perf_counter()
        tempo_total = fim - inicio

        assert len(biblioteca.relatorio_livros_emprestados()) == 600
        # Limite conservador para ambiente academico local.
        assert tempo_total < 8.0
```

```
(venv) C:\Users\mathe\Downloads\trabalhomalerba>pytest -v -k sobrecarga
========================================================= test session starts =========================
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\mathe\AppData\Local\Programs\
cachedir: .pytest_cache
rootdir: C:\Users\mathe\Downloads\trabalhomalerba
collected 24 items / 23 deselected / 1 selected

test_sistema.py::TestSobrecarga::test_sobrecarga_cadastro_e_emprestimo PASSED

=============================================== 1 passed, 23 deselected in 0.04s ======================
```

3.1 Métodos da Classe

Os métodos principais da classe Biblioteca foram testados individualmente, garantindo o correto funcionamento das operações de cadastro, login, empréstimo e devolução.

Campo	Definição
O que será testado	Métodos de cadastro, login, empréstimo, devolução e relatórios
Por que será testado	São operações centrais do domínio e concentram regras de negócio críticas
Resultado esperado	Retornos corretos, exceções apropriadas e consistência de estado
Técnica	Manual (apoio) e automática (pytest)
Estágio	Unidade
Abordagem	Caixa branca e caixa preta
Responsáveis	Desenvolvedor e equipe de testes

3.2 Persistência/consistência de dados

Os dados são mantidos em memória durante a execução, garantindo consistência entre usuários, livros e empréstimos.

Campo	Definição
O que será testado	Integridade do estado após sequências de empréstimo/[redacted]devolução
Por que será testado	Evitar inconsistências de disponibilidade e duplicidade de operação
Resultado esperado	Estados coerentes após cada transição de fluxo
Técnica	Manual e automática
Estágio	Sistema
Abordagem	Caixa preta

Campo	Definição
Responsáveis	Equipe de testes

3.3 Integração de componentes

Foi validada a integração entre os módulos, assegurando que ações como empréstimo e devolução atualizam corretamente o estado do sistema.

Campo	Definição
O que será testado	Fluxos entre módulos (Usuário, Acervo, Circulação)
Por que será testado	Defeitos de integração causam falhas mesmo com unidade aprovada
Resultado esperado	Fluxos ponta a ponta executados sem quebra e com dados corretos
Técnica	Manual e automática
Estágio	Integração
Abordagem	Caixa preta e caixa branca
Responsáveis	Desenvolvimento + Testes

3.4 Tempo de Resposta

As operações principais apresentaram tempo de resposta adequado, dentro do limite definido de até 2 segundos.

Campo	Definição
Objetivo	Medir se operações críticas (login, buscar livros, emprestar, devolver, consultar relatórios) respondem em até 2 segundos em ambiente local de teste com massa de dados acadêmica (até 1.000 registros em memória).
O que será testado	Tempo de resposta por operação e tempo total de execução de cenários compostos.
Por que será testado	Garantir experiência adequada ao usuário e identificar gargalos antes da entrega.
Resultado esperado	95% das operações críticas com resposta ≤ 2s; nenhuma operação acima de 3s.
Técnica	Automática (scripts de medição/■benchmark simples) e manual (validação exploratória cronometrada).
Estágio do teste	Sistema
Abordagem	Caixa preta
Responsáveis	Equipe de testes e apoio técnico de desenvolvimento

3.5 Animação

Não se aplica ao sistema, por se tratar de aplicação sem interface gráfica.

Campo	Definição
O que será testado	Feedback visual de sucesso/■erro (mensagens, estados de botão, indicadores visuais)
Por que será testado	Melhorar usabilidade e reduzir erro operacional do usuário
Resultado esperado	Usuário recebe retorno visual imediato e compreensível após cada ação
Técnica	Manual

Campo	Definição
Estágio	Sistema/■Aceitação
Abordagem	Caixa preta
Responsáveis	QA e avaliador funcional

Campo	Definição
O que será testado	Legibilidade, contraste básico, textos de erro e confirmação
Por que será testado	Garantir compreensão do fluxo por diferentes perfis de usuário
Resultado esperado	Mensagens claras, sem ambiguidade e com orientação de correção
Técnica	Manual
Estágio	Sistema/■Aceitação
Abordagem	Caixa preta
Responsáveis	QA funcional

4. Recursos

• Documento de requisitos revisado.

• Ambiente de testes disponível e configurado.

• Build funcional com sistema.py e test_sistema.py.

• Massa de dados de teste definida

• Execução de 100% dos casos planejados para o ciclo.

• Registro de evidências e relatório de execução.

• Defeitos críticos corrigidos ou formalmente aceitos.

• Taxa de aprovação dentro da meta definida.

Critério de entrada e saída:

- Sistema implementado e disponível para testes
- Dados de teste preparados

- Todos os testes executados
- Bugs críticos corrigidos ou documentados

Os testes serão considerados aprovados quando os resultados obtidos corresponderem aos resultados esperados definidos em cada caso de teste.

4.1 Ambiente de teste - Software e Hardware

Categoria	Definição
Hardware	Notebook/■desktop padrão acadêmico
Sistema operacional	Windows 10/■11
Linguagem	Python 3.x
Framework de teste	pytest
Interface opcional	React (quando aplicável)
Ferramentas de apoio	IDE, terminal, planilha de controle, PDF

4.2 Ferramenta de teste

Foi utilizado o pytest para a realização dos testes automatizados do sistema, por ser uma ferramenta simples e eficiente para projetos em Python.

Os testes são executados pelo terminal com o comando "pytest -v", permitindo visualizar quais testes passaram ou falharam.

A ferramenta foi utilizada para validar as principais funcionalidades do sistema, além de ajudar na identificação de erros e possíveis regressões após alterações no código.

5. Cronograma

Fase	Duração
Levantamento/■Revisão de requisitos	2 dias
Elaboração do plano de testes	1 dia
Projeto de casos de teste	2 dias
Implementação de testes automatizados	3 dias
Execução de testes manuais	2 dias
Consolidação e documentação final	2 dias
Total estimado	12 dias

• Cobertura dos requisitos críticos: 100%.

• Execução dos casos planejados do ciclo: 100%.

• Taxa de aprovação alvo da suíte automatizada: >= 95%.

• Defeitos críticos abertos ao final: 0 (meta de entrega).

6.1 Casos de Testes

CT-001:

ID do teste: CT-001

Nome: Cadastro de usuário válido

Descrição: Verificar se o sistema cadastra usuário com dados válidos.

Pré-condições: Sistema iniciado; e-mail ainda não cadastrado.

Passos:

Informar nome válido, e-mail válido e senha com 4 ou mais caracteres.

Confirmar cadastro.

Resultado esperado: Usuário cadastrado com ID único e mensagem de sucesso.

CT-002

ID do teste: CT-002
Nome: Login com credenciais válidas
Descrição: Verificar autenticação de usuário previamente cadastrado.
Pré-condições: Usuário existente e ativo no sistema.

Passos:

1. Informar e-mail e senha corretos.
2. Acionar login.

Resultado esperado: Acesso concedido e sessão iniciada.

CT-003

ID do teste: CT-003
Nome: Cadastro de livro válido
Descrição: Verificar inclusão de novo livro no acervo.
Pré-condições: Usuário com permissão para cadastrar livro.

Passos:

1. Informar título e autor válidos.
2. Confirmar cadastro do livro.

Resultado esperado: Livro registrado e disponível para consulta e empréstimo.

CT-004

ID do teste: CT-004
Nome: Empréstimo de livro disponível
Descrição: Validar empréstimo com regras de negócio atendidas.
Pré-condições: Usuário autenticado; livro disponível; usuário com menos de 3 empréstimos ativos.

Passos:

1. Selecionar livro disponível.
2. Confirmar operação de empréstimo.

Resultado esperado: Empréstimo criado; livro marcado como indisponível; prazo de devolução definido.

CT-005

ID do teste: CT-005
Nome: Devolução de livro emprestado
Descrição: Validar devolução de empréstimo ativo.
Pré-condições: Empréstimo ativo existente.

Passos:

1. Localizar empréstimo ativo.
2. Acionar devolução.

Resultado esperado: Empréstimo encerrado e livro novamente disponível.

CT-006

ID do teste: CT-006
Nome: Busca de livros disponíveis
Descrição: Verificar listagem de livros disponíveis no acervo.
Pré-condições: Acervo com livros cadastrados (alguns disponíveis).

Passos:

1. Acessar a funcionalidade de busca de livros.
2. Executar consulta sem filtro e com filtro por título.

Resultado esperado: O sistema retorna apenas livros disponíveis conforme o critério informado.

6.2 Relatório de defeitos (Simulados)

ID: BUG-001
Descrição: Sistema permite empréstimo do mesmo livro para dois usuários em condição de concorrência simulada.

Passos para reproduzir:

1. **Usuário A e Usuário B acessam o mesmo livro disponível quase simultaneamente.**
2. **Ambos confirmam o empréstimo em sequência rápida.**

Resultado esperado: Apenas um empréstimo deve ser aceito; o segundo deve ser bloqueado.

Resultado obtido: Em cenário específico, ambos os empréstimos foram aceitos.

Severidade: Alta
Status: Aberto

Observação: O sistema não possui controle de concorrência, o que pode causar inconsistências em acessos simultâneos.

ID: BUG-002
Descrição: Mensagem de erro de login não orienta correção de credenciais.

Passos para reproduzir:

1. **Informar e-mail válido e senha incorreta.**
2. **Executar login.**

Resultado esperado: Mensagem clara: “Credenciais inválidas. Verifique e-mail e senha.”

Resultado obtido: Mensagem genérica “Erro ao autenticar”.

Severidade: Média
Status: Em correção

ID: BUG-003
Descrição: Relatório de atrasos exibe empréstimo devolvido em cenário específico de atualização tardia.

Passos para reproduzir:

1. **Registrar empréstimo antigo.**
2. **Realizar devolução.**
3. **Executar relatório de atrasos imediatamente após a operação.**

Resultado esperado: Empréstimo devolvido não deve constar no relatório de atraso.

Resultado obtido: O registro aparece temporariamente no relatório.

Severidade: Alta
Status: Aberto

7. Conclusão

Este plano de testes apresenta uma estrutura completa, técnica e rastreável do Sistema de Biblioteca. O documento contempla planejamento, execução e controle de qualidade com foco em requisitos, risco de negócio e evidências objetivas. A adoção de testes unitários, integração, sistema, aceitação, regressão e desempenho fortalece a confiabilidade da solução e sustenta a evolução do software. Como melhoria futura, recomenda-se a implementação de controle de concorrência e persistência em banco de dados.